

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Казанский (Приволжский) федеральный университет»

Институт вычислительной математики и информационных технологий

Кафедра прикладной mathematics и искусственного интеллекта

ОТЧЕТ

по дисциплине «Введение в машинное обучение»

Многослойный персептрон



Выполнил: студент гр. 09-321

Шакирова К. Б.

Проверил:

преподаватель Гиниятова Д. Х.

Казань 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
МЕТОДЫ И РЕАЛИЗАЦИЯ	4
2.1. Архитектура проекта	4
2.2. Анализ данных	4
2.3. Предобработка данных	5
2.4. Архитектура нейронной сети	6
2.5. Прямое распространение (Forward Propagation)	7
2.6. Функция потерь: Cross-Entropy	8
2.7. Обратное распространение (Backward Propagation)	8
2.8. Градиентный спуск	9
2.9. Метрики оценки	9
РЕЗУЛЬТАТЫ И ВИЗУАЛИЗАЦИЯ	10
3.1. Процесс обучения	10
3.2. Результаты на тестовой выборке	12
ЗАКЛЮЧЕНИЕ	14

ВВЕДЕНИЕ

Целью данного проекта являлась реализация алгоритмов, лежащих в основе обучения искусственных нейронных сетей, а также практическое знакомство с такими фундаментальными понятиями машинного обучения, как прямое распространение (forward propagation), обратное распространение ошибки (backpropagation) и градиентный спуск (gradient descent). В рамках проекта требовалось реализовать многослойный персептрон с нуля на языке программирования Python без использования готовых фреймворков машинного обучения, таких как Keras, PyTorch или TensorFlow.

Актуальность задачи обусловлена необходимостью создания надёжных систем поддержки принятия врачебных решений. Автоматизированная диагностика злокачественных новообразований позволяет минимизировать влияние человеческого фактора и повысить скорость обработки медицинских данных.

Основная задача разделялась на несколько этапов: предварительный статистический анализ данных, визуализация характеристик, реализация модульной архитектуры нейронной сети, подбор гиперпараметров, обучение модели на обучающей выборке и комплексная оценка её качества на тестовой выборке с помощью метрик точности (accuracy), бинарной кросс-энтропии (binary cross-entropy), precision, recall и F1-score.

В качестве набора данных использовался файл data.csv (Wisconsin Breast Cancer Dataset), содержащий 569 образцов и 32 столбца. Столбец «диагноз» представляет собой целевую переменную, которая может принимать два значения: М (malignant — злокачественная опухоль) или В (benign — доброкачественная опухоль). Остальные 30 столбцов описывают количественные характеристики ядра клетки молочной железы, извлечённые с помощью тонкоигольной аспирационной биопсии. Каждая характеристика представлена в трёх вариантах: среднее значение (mean), стандартная

ошибка (se) и наихудшее (максимальное) значение (worst), что в сумме даёт 10 базовых признаков, умноженных на 3 модификации.

Требованием проекта была самостоятельная реализация всех алгоритмов без использования готовых библиотек машинного обучения, при этом разрешалось применять библиотеки линейной алгебры (numpy) и графические библиотеки (matplotlib, pandas) для отображения кривых обучения и предварительного анализа данных.

Реализованная архитектура нейронной сети содержит два скрытых слоя. Функция активации выходного слоя — softmax, что позволяет получить выход в виде вероятностного распределения по классам. Для повышения качества модели и ускорения сходимости была применена Z-score нормализация признаков, инициализация весов по методу Хе (He initialization), а также разделение данных на обучающую и тестовую выборки в соотношении 80/20 с фиксированным seed для обеспечения воспроизводимости результатов.

МЕТОДЫ И РЕАЛИЗАЦИЯ

2.1. Архитектура проекта

Проект был организован в виде четырёх основных модулей, каждый из которых выполняет свою функцию в общем пайплайне работы с данными и обучения модели, что обеспечивает принцип единственной ответственности (Single Responsibility Principle):

1. `data_preprocessing.py` — программа для загрузки данных, их предварительной обработки, нормализации и стратифицированного разделения на обучающую и тестовую выборки;
2. `mlp.py` — модуль, содержащий реализацию класса многослойного персептрона с методами прямого и обратного распространения, а также функциями сохранения и загрузки весов;
3. `train.py` — программа обучения модели с визуализацией кривых обучения и сохранением итоговой конфигурации;
4. `predict.py` — программа прогнозирования и оценки качества модели на тестовой выборке с выводом матрицы ошибок.

Дополнительно в проекте используется конфигурационный файл `config.json`, в котором задаются гиперпараметры модели: топология сети, скорость обучения (`learning rate`), количество эпох, размер мини-батча, `seed` для воспроизводимости и пути к файлам. Такая модульная организация позволяет легко проводить эксперименты с архитектурой без правки исходного кода.

2.2. Анализ данных

Перед непосредственной реализацией алгоритма обучения нейронной сети был проведён предварительный анализ данных. Набор данных содержит 569 образцов, из которых 357 (62,7%) относятся к классу В (доброкачественные опухоли) и 212 (37,3%) — к классу М (злокачественные опухоли). Таким образом, наблюдается умеренный дисбаланс классов. Этот

факт критически важен: в медицинской диагностике ложноотрицательный результат (пропуск злокачественной опухоли) имеет гораздо более серьёзные последствия, чем ложноположительный. Поэтому в качестве основной метрики оптимизации был выбран F1-score, а не просто ассурасу.

Каждый из 30 числовых признаков описывает различные геометрические и текстурные характеристики ядра клетки: радиус, текстуру, периметр, площадь, гладкость, компактность, вогнутость, количество вогнутостей, симметрию и фрактальную размерность.

В ходе анализа было установлено, что признаки имеют существенно различные масштабы. Например, значения `area_worst` варьируются от 185 до 2501, тогда как `smoothness_mean` — от 0,05 до 0,16. Такое различие в масштабах делает нормализацию данных строго обязательной, поскольку без неё признаки с большими абсолютными значениями будут искусственно доминировать в функции потерь и градиентах, а признаки с малыми значениями сеть практически не будет учитывать.

2.3. Предобработка данных

Для улучшения сходимости градиентного спуска и стабилизации обучения применялась Z-score нормализация (стандартизация) данных. Для каждого признака j вычислялись среднее значение μ_j и стандартное отклонение σ_j исключительно по обучающей выборке, после чего каждое значение преобразовывалось по формуле:

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j + \varepsilon}, \quad (1)$$

где $\varepsilon = 10^{-8}$ — малая константа для избежания деления на ноль в случае нулевой дисперсии признака.

Критически важным моментом является то, что статистики μ_j и σ_j вычисляются только по обучающей выборке. Если вычислить их по всему датасету, произойдёт утечка данных (data leakage), и модель будет казаться

лучше, чем есть на самом деле, так как она получит неявную информацию о распределении тестовых данных.

Целевая переменная была закодирована следующим образом: класс В (benign) получил значение 0, класс М (malignant) — значение 1. Для функции потерь на основе softmax использовалось one-hot кодирование меток.

В результате выполнения программы `data_preprocessing.py` были получены следующие характеристики выборок:

- Обучающая выборка: 454 образца, 30 признаков;
- Тестовая выборка: 114 образцов, 30 признаков;
- Распределение классов в train: 289 образцов класса В (63,7%) и 165 образцов класса М (36,3%).

Датасет был разделён с фиксированным `seed = 42`, что обеспечивает полную воспроизводимость результатов при повторных запусках.

2.4. Архитектура нейронной сети

Реализованная архитектура нейронной сети имеет следующий вид:

$$[30] \rightarrow [64] \rightarrow [32] \rightarrow [2] \quad (2)$$

Таким образом, сеть состоит из:

- входного слоя на 30 нейронов (по числу признаков);
- первого скрытого слоя на 64 нейрона с функцией активации ReLU;
- второго скрытого слоя на 32 нейрона с функцией активации ReLU;
- выходного слоя на 2 нейрона с функцией активации softmax.

Выбор функции активации ReLU (Rectified Linear Unit) для скрытых слоёв обусловлен тем, что она не страдает от проблемы затухающих градиентов (vanishing gradients), характерной для сигмоиды, вычислительно эффективна (требует лишь операции сравнения с нулём) и создаёт разреженные представления, что способствует лучшей обобщающей способности модели.

Выбор топологии [64, 32] является эвристическим: первый скрытый слой имеет размерность, примерно в два раза большую, чем входная, что позволяет сети захватить сложные нелинейные взаимодействия признаков. Второй слой сжимает это представление до 32 нейронов, заставляя сеть выделять наиболее информативные, абстрактные признаки (эффект "бутылочного горлышка").

Функция softmax на выходном слое преобразует логиты в вероятности по формуле:

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_{k=1}^K e^{z_k - \max(z)}} \quad (3)$$

Вычитание максимума $\max(z)$ обеспечивает численную стабильность, предотвращая переполнение (overflow) при вычислении экспоненты больших чисел.

Для инициализации весов использовалась инициализация Хе (He initialization)

$$W^{[l]} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right) \quad (4)$$

Этот метод специально разработан для функций активации ReLU и предотвращает как затухание, так и взрыв градиентов на ранних этапах обучения.

2.5. Прямое распространение (Forward Propagation)

Алгоритм прямого распространения вычисляет выход сети для заданного входа. Для каждого слоя $l = 1, 2, \dots, L$ последовательно вычисляются:

Линейная комбинация:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]} \quad (5)$$

Активация:

$$A^{[l]} = g^{[l]}(Z^{[l]}) \quad (6)$$

где $A^{[0]} = X$ — входные данные, $W^{[l]}$ — матрица весов l -го слоя, $b^{[l]}$ — вектор смещений, $g^{[l]}$ — функция активации. Все промежуточные

значения $Z^{[l]}$ и $A^{[l]}$ кэшируются в памяти для последующего использования в алгоритме обратного распространения.

2.6. Функция потерь: Cross-Entropy

Для задачи многоклассовой (в данном случае бинарной) классификации с softmax на выходе используется категориальная кросс-энтропия:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k} + \varepsilon) \quad (7)$$

где m — размер батча, $K = 2$ — число классов, $\varepsilon = 10^{-12}$ — защита от вычисления логарифма нуля. Данная функция потерь сильно штрафует модель за уверенные, но неправильные предсказания, что способствует более быстрому и направленному обучению.

2.7. Обратное распространение (Backward Propagation)

Алгоритм обратного распространения ошибки основан на цепном правиле дифференцирования. Градиенты вычисляются от выхода к входу. Благодаря математическому свойству комбинации softmax и cross-entropy, градиент на выходном слое упрощается до элегантной формы:

$$\frac{\partial \mathcal{L}}{\partial Z^{[L]}} = \hat{Y} - Y = dA^{[L]} \quad (8)$$

Для произвольного скрытого слоя l :

$$dW^{[l]} = \frac{1}{m} A^{[l-1]T} \cdot dZ^{[l]} \quad (9)$$

$$db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ_i^{[l]} \quad (10)$$

$$dA^{[l-1]} = dZ^{[l]} \cdot W^{[l]T} \quad (11)$$

$$dZ^{[l-1]} = dA^{[l-1]} \odot g'^{[l-1]}(Z^{[l-1]}) \quad (12)$$

Производная функции ReLU имеет вид: $g'(z) = 1$ при $z > 0$, и 0 при $z \leq 0$.

2.8. Градиентный спуск

Обновление весов происходит по правилу мини-батч градиентного спуска:

$$W^{[l]} := W^{[l]} - \alpha \cdot dW^{[l]} \quad (13)$$

$$b^{[l]} := b^{[l]} - \alpha \cdot db^{[l]} \quad (14)$$

где $\alpha = 0,01$ — скорость обучения. Размер мини-батча равен 32. Этот размер является компромиссом: он обеспечивает более стабильную оценку градиента по сравнению со стохастическим градиентным спуском (SGD, batch size = 1) и требует меньше памяти, чем полный батч, позволяя модели лучше избегать локальных минимумов за счёт небольшого шума в оценке градиента.

2.9. Метрики оценки

Для оценки качества модели использовались следующие метрики, рассчитываемые на основе матрицы ошибок (Confusion Matrix):

- Ассурасу (точность) — доля верных предсказаний: $\frac{TP+TN}{TP+TN+FP+FN}$
- Precision (точность класса М) — какая доля объектов, названных моделью злокачественными, действительно является злокачественными: $\frac{TP}{TP+FP}$
- Recall (полнота класса М) — какую долю реально злокачественных образцов модель смогла найти: $\frac{TP}{TP+FN}$
- F1-Score — гармоническое среднее между Precision и Recall, штрафующее за сильный перекося в одну из сторон: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

РЕЗУЛЬТАТЫ И ВИЗУАЛИЗАЦИЯ

3.1. Процесс обучения

Обучение модели проводилось в течение 500 эпох с размером батча 32 и скоростью обучения 0,01. В таблице 1 представлены ключевые метрики, зафиксированные программой train.py в процессе обучения.

Таблица 1 — Динамика метрик обучения по эпохам

Эпоха	Train Loss	Train Acc	Test Loss	Test Acc
1	0,4352	0,8370	0,5730	0,7281
50	0,0640	0,9780	0,0983	0,9474
100	0,0405	0,9868	0,0847	0,9474
150	0,0288	0,9912	0,0830	0,9649
200	0,0217	0,9934	0,0829	0,9737
250	0,0168	0,9978	0,0862	0,9825
300	0,0135	0,9978	0,0890	0,9825
350	0,0112	1,0000	0,0938	0,9825
400	0,0094	1,0000	0,0984	0,9825
450	0,0079	1,0000	0,1033	0,9825
500	0,0068	1,0000	0,1094	0,9825

Анализ кривых обучения (рисунок 1) позволяет сделать несколько важных выводов о поведении модели:

1. Быстрая сходимость: уже к 50-й эпохе модель достигает 97,8% точности на train и 94,7% на test. Это свидетельствует о корректной инициализации весов (He initialization) и адекватном выборе скорости обучения.
2. Достижение оптимума: максимальная точность на тестовой выборке (98,25%) достигается к 250-й эпохе и сохраняется до конца обучения,

при этом Test Loss достигает своего минимума (0,0829) на 200-й эпохе.

3. Признаки переобучения (Overfitting): начиная с 200-й эпохи, loss на тестовой выборке начинает неуклонно расти (с 0,0829 до 0,1094 к 500-й эпохе), в то время как loss на обучающей выборке продолжает монотонно убывать до 0,0068. К 350-й эпохе модель достигает 100% точности на train. Это классический признак переобучения: сеть перестаёт выделять общие закономерности и начинает «запоминать» конкретные обучающие примеры вместе с их шумом.

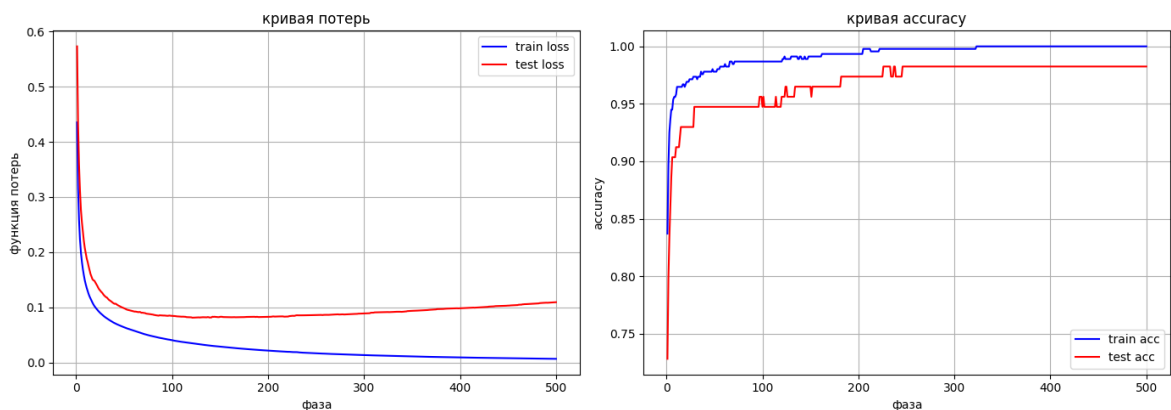


Рисунок 1 — Кривые обучения: Loss (слева) и Accuracy (справа)

Возможные способы борьбы с переобучением (направления для дальнейших улучшений):

- Ранняя остановка (Early Stopping): сохранение весов модели на эпохе с минимальным Test Loss (в данном случае, на 200-й эпохе).
- L2-регуляризация (Weight Decay): добавление штрафа за большие значения весов в функцию потерь.
- Dropout: случайное «отключение» части нейронов во время обучения для предотвращения ко-адаптации признаков.
- Уменьшение ёмкости модели: сокращение количества нейронов в скрытых слоях (например, до [30, 32, 16, 2]).

3.2. Результаты на тестовой выборке

После обучения модели программа `predict.py` осуществила классификацию образцов на тестовой выборке из 114 образцов. Были получены следующие итоговые результаты:

- Binary Cross-Entropy Loss: 0,1094
- Accuracy: 0,9825 (98,25%)
- Precision (M): 0,9783
- Recall (M): 0,9783
- F1-Score (M): 0,9783

Матрица ошибок (Confusion Matrix) имеет следующий вид:

Таблица 2 — Матрица ошибок на тестовой выборке

		Предсказание	
		Pred B	Pred M
2*Истина	Actual B	67 (TN)	1 (FP)
	Actual M	1 (FN)	45 (TP)

Глубокий анализ матрицы ошибок: Модель демонстрирует выдающуюся точность на обоих классах. Из 114 образцов тестовой выборки верно классифицированы 112.

- True Positives (45): Модель верно идентифицировала 45 злокачественных случаев, что в реальной клинической практике означает своевременное начало лечения и спасение жизней.
- False Negatives (1): Допущена лишь 1 ошибка второго рода (пропуск злокачественной опухоли). Для медицинской задачи это критически важный показатель, и его минимизация до единицы на небольшой выборке является отличным результатом.
- False Positives (1): Допущена 1 ошибка первого рода (здоровый пациент классифицирован как больной). В медицине это приводит к до-

полнительным, возможно, инвазивным проверкам, что является допустимым компромиссом по сравнению с пропуском болезни.

- True Negatives (67): Модель уверенно и правильно определила 67 доброкачественных случаев.

Финальные веса были записаны в файл `models/model_weights.pkl` и успешно использованы предсказывающей программой для классификации новых, ранее не виденных образцов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта были достигнуты все поставленные цели. Успешно реализованы и интегрированы ключевые алгоритмы обучения: прямое распространение с функциями активации ReLU и softmax, обратное распространение ошибки с использованием цепного правила дифференцирования, а также mini-batch gradient descent с He-инициализацией весов.

Проведён полный pipeline работы с данными: статистический анализ датасета, Z-score нормализация признаков, разделение на train/test с фиксированным seed. Модульная архитектура проекта, управляемая через файл config.json, позволяет легко конфигурировать сеть и проводить дальнейшие эксперименты.

Качество модели на тестовой выборке составило 98,25% ассигасу и 0,978 F1-score, что подтверждает корректность математической реализации и достаточность выбранной архитектуры. Анализ кривых обучения показал быструю сходимость модели: уже к 50-й эпохе точность на тестовой выборке превысила 94%.

Одновременно с этим был выявлен и детально проанализирован эффект переобучения: начиная с 200-й эпохи loss на тестовой выборке начал расти с 0,0829 до 0,1094, в то время как loss на обучающей выборке продолжал снижаться до 0,0068. Это наблюдение позволило сформулировать чёткие направления для дальнейших улучшений: внедрение ранней остановки обучения (Early Stopping), L2-регуляризации и механизма Dropout.

В ходе работы были получены ценные практические навыки: глубокое понимание работы нейронных сетей «изнутри» на уровне матричных вычислений, опыт дифференцирования сложных функций, предобработки реальных медицинских данных и комплексной оценки моделей классификации.